Project Description – McQueen

McQueen is a real-time vehicle communication platform that allows cars on a network to share safety-critical information with each other instantly. Every vehicle continuously broadcasts its location, speed, heading, and status to a central server, which processes the data and pushes warnings back out to all connected vehicles in under 50 milliseconds.

The system detects three types of safety events. When two vehicles are on a collision course, McQueen calculates the time to impact using real geospatial math and fires a warning to both drivers before they are physically close enough to react manually. When a vehicle brakes hard, every vehicle behind it is alerted immediately — even those too far back to see the hazard. When a crash is detected, an SOS is broadcast to the entire network and emergency services are notified automatically.

McQueen is built on a Node.js and Socket.io backend for real-time communication, a vehicle simulator that drives five cars along real GPS routes through a scripted demo scenario, and a React dashboard with a live Mapbox map showing every vehicle moving in real time, colour-coded by their current safety status, alongside a live alert feed.

The core value proposition is reaction time. A human driver reacts in approximately 1.5 seconds. McQueen reacts in under 50 milliseconds — fast enough to warn drivers of dangers they cannot yet see, and to coordinate emergency response without any manual input from anyone involved.

Team Overview

- Person 1 — Backend Dev A: Server setup, Socket.io, event handling, deployment
- Person 2 — Backend Dev B: Collision detection, brake hazard logic, SOS, alert system
- Person 3 — Simulator Dev: Vehicle movement, GPS waypoints, scripted demo events
- Person 4 — Frontend Dev: Map, markers, alert feed, dashboard UI, pitch slides
-

Telemetry Payload (sent by simulator every 500ms per vehicle)

- id — string, e.g. car_01
- lat — number, latitude coordinate
- lng — number, longitude coordinate
- speed — number, km/h
- heading — number, degrees 0-360
- braking — boolean
- status — string: "normal" | "braking" | "crash"

Alert Payload (sent by server when a safety event is detected)

- type — string: "COLLISION_WARNING" | "BRAKE_HAZARD" | "SOS"
- vehicleId — string, the vehicle that triggered the alert
- message — string, human-readable description
- location — object: { lat, lng }
- timestamp — number, Unix milliseconds

Socket.io Event Names — Everyone Must Use These Exact Names

- telemetry — vehicle position update, sent by simulator
- alert — any warning event, sent by server to all clients
- brake_hazard — braking event, sent by simulator to server
- sos — crash detected, sent by simulator to server
- reset — restart demo scenario, sent by frontend to server

Note: Person 1 creates a file called events.js with these as exported constants. Everyone else imports from it. This prevents typos.

Person 1 — Backend Dev A

Role: You build and own the server. Everything else depends on your server being up and working. You are also the integration lead — when two pieces need to connect, you coordinate it.

Files You Own

- server.js — Express + Socket.io setup, all event handlers
- vehicles.js — in-memory vehicle state store
- events.js — shared Socket event name constants, create this first and share with everyone
- package.json — all dependencies and npm scripts

Setup Commands

- mkdir v2x-backend && cd v2x-backend
- npm init -y
- npm install express socket.io cors
- npm install -D nodemon
- Add to package.json scripts: "dev": "nodemon server.js" and "start": "node server.js"

What You Build — Hour by Hour

Hour 0:00 to 0:30 — Project Setup

- Initialise the Node project with the commands above
- Create events.js with all Socket event name constants as exported strings
- Share events.js with the whole team immediately via Google Doc or message
- Confirm server starts on port 3000 with no errors

Hour 0:30 to 1:30 — Server Skeleton

- Create server.js with Express and Socket.io wired together
- Add a GET / route that returns { status: 'V2X Server Online' }
- Add socket.on('connection') handler that logs each new connection
- Test: open two browser tabs, confirm both connections are logged in the terminal

Hour 1:30 to 2:30 — Vehicle State Store

- Create vehicles.js with an in-memory object to track all vehicles
- Export four functions: update(vehicle), getAll(), remove(id), count()

- Import vehicles.js into server.js
- On receiving a telemetry event: call vehicles.update(data), then broadcast the data back out to all connected clients with io.emit
- Log all incoming telemetry to console so Person 2 and Person 3 can verify their connections

Hour 2:30 to 3:30 — Non-Collision Alert Handlers

- Handle brake_hazard event: receive data from simulator, emit an alert event to all clients with type BRAKE_HAZARD
- Handle sos event: receive data from simulator, emit an alert event to all clients with type SOS
- Handle reset event: clear the vehicles state object, emit a reset_ack event to all clients
- Test each handler manually using a Socket.io test client or Thunder Client

Hour 3:30 to 5:00 — Integrate Person 2's Collision Logic

- At hour 3:00 Person 2 should be delivering collision.js — import checkCollisions() from it
- On every incoming telemetry event, call checkCollisions(data, vehicles.getAll()) after updating the state
- If checkCollisions returns any alerts, emit each one as an alert event to all clients
- Add alert suppression: track the last time each collision pair fired in a recentAlerts object, only emit if it has been more than 3 seconds since the last alert for that pair
- Test: confirm collision alerts appear in the browser console when Person 3's simulator is running

Hour 5:00 to 6:00 — Deploy to Railway

- Push the backend to a GitHub repository
- Go to railway.app, create a new project, and connect to the GitHub repo
- Railway auto-detects Node.js — set the start command to: node server.js
- Wait for deployment to complete and copy the public URL
- Share the public URL immediately with Person 3 and Person 4
- Test that a connection from a different machine or tab works correctly

Hour 6:00 to 8:00 — Integration Support

- Help Person 3 debug any simulator connection issues
- Help Person 4 debug any frontend Socket.io connection issues
- Monitor server logs during the first end-to-end demo run
- Fix any edge cases that appear — duplicate alerts, stale vehicle state, dropped connections

Hour 8:00 to 10:00 — Demo Hardening

- Run the full demo scenario at least five times end-to-end

- Confirm that reset works cleanly every single run with no leftover vehicle state
- Confirm that the server stays stable for the entire duration with no crashes or restarts
- Be present and monitoring server logs during the judge presentation

Your Acceptance Criteria — Definition of Done

- Server starts locally with no errors by hour 1:00
- events.js shared with all team members by hour 0:30
- Telemetry events received and logged in console by hour 2:00
- Telemetry re-broadcast to all connected clients by hour 2:30
- All three alert types firing correctly by hour 5:00
- Deployed to Railway with working public URL by hour 6:00
- Reset works cleanly on repeated demo runs by hour 8:00
- Full demo runs five consecutive times without any server errors by hour 9:00

Person 2 — Backend Dev B

Role: You own the safety intelligence of the system. While Person 1 handles the server infrastructure, you handle the logic that makes the project scientifically meaningful. Your collision detection math is the core engine of the entire demo.

Files You Own

- collision.js — Haversine formula, time-to-collision calculation, collision alert builder
- alerts.js — alert formatting functions and duplicate suppression logic

What You Build — Hour by Hour

Hour 0:00 to 1:00 — Understand the System and Plan

- Read the shared data contract carefully — memorise the telemetry payload shape
- Sketch the collision detection logic on paper before writing any code
- Set up a local test file with hardcoded fake coordinates to test your math in isolation — you do not need Person 1's server to test this
- Install nothing extra — you are working in pure JavaScript alongside Person 1's Node project

Hour 1:00 to 2:30 — Build collision.js

- Implement haversine(lat1, lng1, lat2, lng2) — takes four numbers, returns distance in metres
  - Use Earth radius R = 6371000 metres
  - Convert degrees to radians: multiply by Math.PI / 180
  - Apply the haversine formula, return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a))
- Implement timeToCollision(car1, car2) — takes two vehicle objects, returns seconds
  - Calculate distance using haversine
  - Calculate closing speed: (car1.speed + car2.speed) / 3.6 to convert km/h to m/s
  - Return distance / closingSpeed, or Infinity if closingSpeed is zero or negative
- Implement checkCollisions(car, allVehicles) — takes a single vehicle and the full state object
  - Loop over all other vehicles
  - For each pair, call timeToCollision and haversine
  - If TTC is less than 3 seconds AND distance is less than 80 metres, add a COLLISION_WARNING alert to the results array
  - Return the array of alert objects
- Test with hardcoded data: two cars 60 metres apart, both at 60 km/h — TTC should be approximately 1.8 seconds, which is below the 3-second threshold and should trigger a warning

Hour 2:30 to 3:30 — Build alerts.js

- Implement buildCollisionAlert(car1Id, car2Id, ttc, distance) — returns a correctly shaped alert object with type COLLISION_WARNING
- Implement buildBrakeAlert(vehicle) — returns a correctly shaped alert object with type BRAKE_HAZARD
- Implement buildSOSAlert(vehicle) — returns a correctly shaped alert object with type SOS
- Implement shouldSuppress(key, recentAlerts, cooldownMs) — returns true if the same alert key has fired within cooldownMs milliseconds
  - key should be the two vehicle IDs sorted and joined, e.g. car_01_car_04
  - Use a plain object to store timestamps: recentAlerts[key] = Date.now()
  - Default cooldown is 3000 milliseconds
  - This prevents the alert feed from being flooded with the same collision warning 10 times per second

Hour 3:30 to 5:00 — Integrate with Person 1's Server

- Hand collision.js to Person 1 so they can import checkCollisions into server.js
- Sit together with Person 1 and test the integration — run Person 3's simulator and watch for collision alerts in the browser console
- Tune the thresholds if needed: adjust the TTC threshold or distance threshold until alerts fire at the right moment in the demo scenario
- Verify that the alert suppression is working — the same car pair should not flood the alert feed

Hour 5:00 to 7:00 — Crash Severity and SOS Polish

- Add a crash severity prediction to collision alerts based on the kinetic energy at impact
  - mass = 1400 kg for an average car
  - closingSpeed in m/s = (car1.speed + car2.speed) / 3.6
  - kineticEnergy = 0.5 * mass * closingSpeed squared
  - If kineticEnergy < 50000: severity is MINOR. If < 200000: MODERATE. Otherwise: SEVERE
- Add severity field to the collision alert object
- Write a brief 2-sentence description of the collision detection approach for Person 4 to include in the pitch slides

Hour 7:00 to 9:00 — Demo Scenario Tuning

- Work with Person 3 to confirm that the collision alert fires at the correct moment in the 40-second demo
- Run the full demo scenario at least five times and watch for any missed or late alerts
- Fix any edge cases — alerts firing too early, alerts not firing at all, suppress logic being too aggressive

Hour 9:00 to 10:00 — Final Checks

- Confirm all three alert types appear correctly in Person 4's dashboard during a full demo run
- Confirm severity field is displayed correctly in the alert feed
- Support the team during rehearsal

The Core Math — For Reference

Haversine Distance

- dLat = (lat2 - lat1) * Math.PI / 180
- dLng = (lng2 - lng1) * Math.PI / 180
- a = sin(dLat/2)^2 + cos(lat1 in radians) * cos(lat2 in radians) * sin(dLng/2)^2
- distance = 6371000 * 2 * atan2(sqrt(a), sqrt(1-a))

Time to Collision

- closingSpeed = (car1.speed + car2.speed) / 3.6
- ttc = distance / closingSpeed
- Fire COLLISION_WARNING if ttc < 3 AND distance < 80

Your Acceptance Criteria — Definition of Done

- haversine() returns a correct result — test: two points 1 degree of latitude apart should return approximately 111000 metres
- checkCollisions() fires a warning when two cars are 60m apart at 60 km/h each
- Alert suppression prevents more than one alert per 3 seconds for the same car pair
- All three alert builder functions return objects that match the shared alert payload shape
- Severity field is correctly calculated and included in collision alerts
- Collision alert fires at the correct moment during the scripted demo scenario

Person 3 — Simulator Dev

Role: You create the data that drives the entire system. Without a working simulator, there is nothing for the server to process and nothing for the map to display. Your most important single deliverable is the scripted 40-second demo scenario that plays identically every time a judge is watching.

Files You Own

- simulator.js — main entry point, creates and starts all vehicle instances
- vehicle.js — Vehicle class with movement, state, and broadcast logic
- routes.js — GPS waypoint arrays for all five vehicles
- scenario.js — scripted demo event timeline with setTimeout chain

What You Build — Hour by Hour

Hour 0:00 to 1:30 — Vehicle Engine

- Create a Vehicle class in vehicle.js
  - Constructor takes: id (string), route (array of {lat, lng} objects), speed (km/h)
  - Properties: id, lat, lng, speed, heading, braking (false), status ('normal'), currentWaypointIndex (0)
- Implement a move() method that advances the vehicle toward its next waypoint
  - When the vehicle reaches within a small threshold of a waypoint, increment currentWaypointIndex
  - If the last waypoint is reached, loop back to the beginning
  - Update lat and lng each call by interpolating toward the target waypoint
- Implement a broadcast(socket) method that calls socket.emit('telemetry', payload) with the correct telemetry shape from the shared data contract
- Test a single vehicle instance: create it with two hardcoded waypoints and log the position every 500ms to confirm it moves

Hour 1:30 to 2:30 — Define GPS Routes

- Open Google Maps and manually note lat/lng coordinates for each vehicle route
- Pick a real city area — Midtown Manhattan is a good choice since intersections are on a clear grid
- Create routes.js exporting route arrays for car_01 through car_05
- car_01: 3 to 4 waypoints heading north — this car will crash at its third waypoint
- car_02: 3 to 4 waypoints heading east — this car will brake hard midway through its route
- car_03: 3 to 4 waypoints heading south — this car drives normally with no special events
- car_04: 3 waypoints heading east along a street — must pass through the same intersection as car_05

- car_05: 3 waypoints heading west along the same street — must be on a head-on course with car_04

Note: car_04 and car_05 MUST come within 80 metres of each other at around the T+20s mark. Open Google Maps and measure to confirm their routes actually intersect.

Hour 2:30 to 3:30 — Connect to the Server

- Install socket.io-client: npm install socket.io-client
- In simulator.js, connect to Person 1's server: const socket = io('http://localhost:3000')
- Create all five Vehicle instances with their routes from routes.js
- Start a setInterval that calls move() and broadcast(socket) on each vehicle every 500ms
- Confirm that telemetry messages appear in Person 1's server console
- Confirm all five cars are visible in Person 4's map once the frontend is connected

Hour 4:00 to 6:00 — Scripted Demo Events

- Create scenario.js and export a runDemoScenario(vehicles, socket) function
- The function uses a chain of setTimeout calls to trigger events at exact times
- T+10 seconds: set car_02.braking = true and car_02.speed = 10, then call socket.emit('brake_hazard', car_02 telemetry)
- T+20 seconds: set car_04.speed = 80 and car_05.speed = 80 so they accelerate toward their intersection point
  - The collision detection on Person 2's server should fire automatically once they are within 80 metres at this speed
- T+35 seconds: set car_01.status = 'crash' and car_01.speed = 0, then call socket.emit('sos', car_01 telemetry)
- Import and call runDemoScenario from simulator.js, triggered when the server emits a start_demo event or a local button is pressed

Hour 6:00 to 7:30 — Reset Functionality

- Implement a resetScenario(vehicles) function in scenario.js
- Reset restores all vehicles to their starting waypoint positions
- Reset sets all vehicles: status back to 'normal', braking back to false, speed back to their original default speed
- Listen for the reset event from the server and call resetScenario when it fires
- Test: run the full demo, then reset, then run the demo again — confirm the second run is identical to the first
- Run this cycle five consecutive times to confirm there is no state leakage between runs

Hour 7:30 to 10:00 — Timing and Rehearsal

- Adjust the setTimeout delays until the 40-second story feels natural — not too fast, not too slow
- Add console.log statements before each scripted event so the team can see timing during rehearsal

- Work with Person 4 to confirm that map colour changes align with your event timing
- Work with Person 2 to confirm that the collision warning fires at the right moment
- Rehearse the full demo at least five times with the whole team

The 40-Second Demo Timeline

T+0s — Five vehicles start moving on the map

- All five green markers appear and begin moving along their routes
- Telemetry broadcasting at 500ms intervals

T+10s — car_02 brakes hard

- car_02 speed drops to 10 km/h, braking set to true
- socket.emit('brake_hazard') fires
- Server broadcasts BRAKE_HAZARD alert to all clients
- Dashboard: car_02 turns orange, alert appears in the feed

T+20s — car_04 and car_05 on collision course

- Both cars speed up to 80 km/h and their routes converge
- Person 2's collision detection fires COLLISION_WARNING when they are within 80m
- Dashboard: both cars flash red, collision alert appears with time-to-impact

T+35s — car_01 crashes

- car_01 status set to 'crash', speed set to 0
- socket.emit('sos') fires
- Server broadcasts SOS alert to all clients
- Dashboard: car_01 goes dark, pulsing red circle appears, SOS alert in feed

Your Acceptance Criteria — Definition of Done

- All five vehicles broadcast valid telemetry to the server every 500ms
- car_04 and car_05 come within 80 metres of each other at approximately T+20s
- Brake hazard event fires at T+10s with car_02 speed below 15 km/h
- SOS event fires at T+35s with car_01 status set to 'crash'
- Reset restores all vehicles to start positions in under one second with no stale state
- The full demo can be run five consecutive times with identical results each time

Person 4 — Frontend Dev

Role: You own everything judges actually see. The map dashboard is the project from a judge's perspective. You also own the pitch — the narrative that makes the demo land emotionally, not just technically. Visual clarity and a confident 40-second story are worth as much as the code itself.

Files You Own

- src/App.jsx — root layout, Socket.io connection, global state
- src/socket.js — Socket.io client singleton
- src/components/Map.jsx — Mapbox map and all vehicle markers
- src/components/AlertFeed.jsx — scrolling alert sidebar
- src/components/StatusBar.jsx — vehicle count, warning count, network status

Setup Commands

- npm create vite@latest v2x-frontend -- --template react
- cd v2x-frontend && npm install
- npm install socket.io-client mapbox-gl lucide-react
- npm install -D tailwindcss postcss autoprefixer
- npx tailwindcss init -p

Note: Create a Mapbox account before the hackathon and save your API key. Do not waste time on this during the clock.

What You Build — Hour by Hour

Hour 0:00 to 1:30 — Project Setup and Map

- Run the setup commands above
- Create src/socket.js — a single exported Socket.io client instance pointing to Person 1's server URL
- In App.jsx: import the socket, set up a useEffect to listen for events, store vehicle data in useState
- In Map.jsx: render a Mapbox GL map centred on your demo city using your API key
- Confirm the map tiles load with no errors in the browser console

Hour 1:30 to 3:00 — Live Vehicle Markers

- In App.jsx: listen to the telemetry socket event and update a vehicles object in state — keyed by vehicle ID
- In Map.jsx: render one Mapbox Marker for each vehicle in the vehicles state
- On each telemetry event, update the marker's position using marker.setLngLat([lng, lat])

- Confirm that all five car markers appear on the map and visibly move when the simulator is running

Hour 3:00 to 5:00 — Alert-Driven Visual Changes

- In App.jsx: listen to the alert socket event and store alerts in an array in state
- Track each vehicle's current alert status in a separate object in state — updated whenever an alert arrives for that vehicle
- In Map.jsx: change the colour of each marker based on the vehicle's current status
  - Normal: green — hex #22c55e
  - Brake hazard nearby: orange — hex #f97316
  - Collision warning: red — hex #ef4444
  - Crashed: dark grey — hex #1f2937
- On an SOS alert: add a pulsing red circle at the crash coordinates using a Mapbox circle layer with a CSS pulse animation
- Test by triggering each event type and confirming the correct colour change appears on the map

Hour 5:00 to 6:30 — Alert Feed Sidebar

- Build AlertFeed.jsx as a vertically scrolling list of alert cards
- Each new alert slides in from the top using a CSS transition
- Each alert card shows: alert type label, affected vehicle ID, message text, and timestamp
- Colour-code the left border of each card by type: red for COLLISION_WARNING, orange for BRAKE_HAZARD, dark red for SOS
- Cap the visible feed at 10 alerts — older alerts scroll out of view
- Render AlertFeed in App.jsx alongside the map — map on the left, feed on the right

Hour 6:30 to 7:30 — Status Bar

- Build StatusBar.jsx as a horizontal bar across the top of the dashboard
- Show: total active vehicle count, number of active warnings, and network connection status (Online or Offline)
- Calculate latency by comparing the timestamp in each telemetry event to Date.now() — display the most recent value in milliseconds
- Update all values reactively as new socket events arrive

Hour 7:30 to 8:30 — Run Demo Button and Polish

- Add a Run Demo button that emits start_demo to the server and triggers Person 3's scenario
- Add a Reset button that emits reset to the server and clears all alerts from the local state
- Keep the layout simple: dark background, white text, map taking up roughly two thirds of the screen width, alert feed in the remaining third
- Confirm the full dashboard looks correct in a full-screen browser window on a laptop

Hour 8:30 to 9:30 — Pitch Slides

- Create five slides in Canva — keep each slide to one main idea with minimal text
- Slide 1 — The Problem: 94% of road accidents are caused by human error. 1.35 million deaths per year globally. The average human reaction time is 1.5 seconds.
- Slide 2 — The Solution: V2X — vehicles that warn each other in real time. Our system reacts in under 50 milliseconds. No driver input required.
- Slide 3 — Live Demo: This slide just says 'Let us show you.' Switch to the browser for the demo.
- Slide 4 — How It Works: A simple diagram of the four components — Simulator, Server, Dashboard, and the Socket.io connections between them. Draw this in Excalidraw and paste it in.
- Slide 5 — What's Next: Integration with real C-V2X hardware modules. The software architecture is hardware-agnostic. Phase two is feeding hazard data directly into city traffic management systems.

Hour 9:30 to 10:00 — Rehearsal

- Run the full demo at least three times with the whole team watching
- Practise saying the narration script below alongside the live demo
- Prepare answers for the two most likely judge questions

The Narration Script — Memorise This

Say these words while the demo plays. Each line corresponds to a moment in Person 3's scenario.

At the start:

- 'Here are five vehicles on the V2X network, broadcasting their position and speed twice a second to every other vehicle on the road.'

At T+10s when car_02 brakes:

- 'Car 2 has detected a road hazard and braked hard. Every vehicle behind it has been warned instantly — before they can even see the hazard. That warning travelled in under 50 milliseconds.'

At T+20s when collision warning fires:

- 'Car 4 and Car 5 are now on a collision course. Our system has calculated time-to-impact at 2.1 seconds and alerted both drivers simultaneously. No human reaction time involved.'

At T+35s when SOS fires:

- 'Car 1 has crashed. An SOS has been broadcast to the entire network in real time. Every vehicle within one kilometre is warned. Emergency services are notified. And not a single driver had to make a phone call.'

Answers to Expected Judge Questions

Question: Isn't this just a simulation?

- 'Yes — the vehicles are simulated. But the communication architecture is production-ready. This same Socket.io layer maps directly onto C-V2X radio hardware. We are demonstrating the software stack that sits on top of real DSRC or 5G modules. The simulation is the demo environment, not the system itself.'

Question: How would you scale this?

- 'We replace Socket.io with an MQTT broker for the radio layer and connect to real on-board hardware units. The server logic — the collision detection, the alert routing — stays identical. City traffic management systems can subscribe to the same hazard feed as individual vehicles.'

Your Acceptance Criteria — Definition of Done

- All five vehicle markers appear on the Mapbox map within two seconds of the simulator starting
- Marker colours update within 200ms of receiving an alert for that vehicle
- Alert feed displays new alerts with slide-in animation within 100ms of receipt
- Status bar shows correct vehicle count and live latency
- Run Demo and Reset buttons work reliably across five consecutive demo runs
- Dashboard renders correctly in full-screen browser on a 1080p laptop
- All five pitch slides are complete before hour 9:30
- Narration script is memorised and rehearsed

Team Sync Points and Integration Milestones

These are the moments when individual work needs to connect. Plan for these explicitly.

Hour 0:00 — Before Anyone Writes Code

- Person 1 creates events.js and shares it with the team
- Everyone picks the same demo city and writes down the GPS coordinates area to use
- Create a shared Google Doc with the telemetry payload shape written

Hour 3:00 — Simulator Connects to Server

- Person 3 points the simulator at Person 1's local server
- Person 1 and Person 3 confirm together that telemetry is arriving in the server console
- If there are field name mismatches, fix them now before Person 2 and Person 4 also depend on the format

Hour 4:00 — Collision Alerts and Map Working

- Person 2 hands collision.js to Person 1 — they integrate and test together
- Person 4 connects the frontend to Person 1's server for the first time
- Both of these should happen at roughly the same time — by the end of hour 4 you should have cars moving on the map and alerts appearing in the browser console

Hour 6:00 — Full Dashboard Connected

- Person 4's alert feed is receiving and displaying live alerts from the server
- Person 1 has deployed to Railway and shared the public URL
- The whole system should be working end-to-end at this point, even if the UI is not polished yet

Hour 8:00 — First Full Demo Run

- All four team members watch the full 40-second scenario play out together for the first time
- Note any timing issues, visual glitches, or missing alerts
- Fix anything critical — polish everything else

Hour 9:00 — Rehearsal

- Run the demo at least three more times
- Person 4 practises the narration script
- Every team member should know the basic story of the demo well enough to describe it if asked by a judge

If Something Goes Wrong

Railway deployment fails:

- Run: ngrok http 3000 — this gives you a public URL in under 10 seconds
- Install ngrok before the hackathon and have an account ready

Collision alerts not firing:

- Test collision.js in isolation with hardcoded coordinates first — rule out the math before blaming the server
- Temporarily hardcode a scenario where TTC is definitely below 3 seconds to confirm the event pipeline works

Map markers not moving:

- Check that socket.js is pointing at the correct server URL
- Check the browser console for CORS errors — Person 1's server must allow all origins

No time for pitch slides:

- Skip slides entirely and narrate over the live demo — this is actually more impressive than a slide deck with a broken demo

Pre-Hackathon Checklist — Do This the Night Before

- Node.js v20 or higher installed — verify with: node --version
- npm installed — verify with: npm --version
- VS Code installed with ESLint and Prettier extensions
- Mapbox account created and API key saved somewhere accessible
- Railway account created and linked to GitHub
- ngrok installed and authenticated
- Vite React starter project created and confirmed running
- Node plus Socket.io hello-world server confirmed running
- Demo city GPS area selected — note down a central lat/lng coordinate
- Shared Google Doc created with the data contract — link sent to all four team members